

RCA / Postmortem — Incident #1: Pod Crash Loop

Field	Value
Incident ID	INC-001
Title	Notes API replica set entering CrashLoopBackOff due to corrupt startup configuration
Severity	SEV-3 (degraded capacity; no user-facing outage due to HA)
Status	Resolved
Author	Vishnu K
Date	2026-08-02
Simulated?	Yes — controlled failure injection for resilience testing
Primary alert	NotesApiPodCrashLooping

1. Summary

A corrupt startup configuration was injected into the `notes-api` Deployment by overriding its container start command with a sequence that exits immediately with a non-zero status. Kubernetes repeatedly attempted to start the new replica set, which failed each time, entering **CrashLoopBackOff** with an exponential backoff climbing to ~5m41s between attempts.

Critically, because the application runs **2 replicas** and Kubernetes uses a rolling update strategy, the original healthy pods continued serving traffic for the entire duration of the incident. **There was no user-facing outage** — high availability held. The incident was detected by the `NotesApiPodCrashLooping` alert and resolved by removing the broken command override.

2. Impact

- User-facing:** None. The healthy replica set (`notes-api-545877b456-*`) continued serving all traffic. HA absorbed the failure.
- Capacity:** Reduced redundancy during the incident — the app was effectively running on a single healthy replica set while the new one crash-looped.
- Data:** No data loss.
- Duration:** ~6 min 20 sec (injection to full recovery).

3. Timeline (UTC)

Time	Event
22:10:10	T+0 — Fault injected via <code>kubectl patch</code> (broken startup command)
22:10:15	New replica set begins cycling Running → Error → CrashLoopBackOff (~5s)

Time	Event
22:10:38	<code>NotesApiPodCrashLooping</code> enters Pending (first restart seen by kube-state-metrics)
22:12:38	Alert transitions to Firing — time-to-detection: ~2m 28s
22:16:19	Recovery initiated (<code>--recover</code> removes the broken command)
22:16:30	Deployment rollout stable; both pods 1/1 Running

4. Detection

- **Primary alert:** `NotesApiPodCrashLooping` — fires when the container restart rate over a 5-minute window exceeds zero, sustained for 2 minutes. Value at firing: **0.0114 restarts/sec**.
- **Secondary alerts observed:**
 - `NotesApiDown` — Prometheus could not scrape the failing pod's `/metrics`.
 - `NotesApiPodNotReady` — the crashing pod never stayed Ready long enough to satisfy its readiness probe (remained Pending during the incident).
- **Time-to-detection:** ~2 min 28 sec from injection to alert firing.

Detection latency breakdown (a deliberate design tradeoff)

The ~2.5-minute detection floor is a composite of three intentional design choices:

Component	Contribution
kube-state-metrics scrape interval	~30s before the first restart is visible
PromQL <code>rate([5m])</code> window	needs multiple samples to compute a non-zero rate
Alert <code>for: 2m</code> clause	deliberate suppression of transient restart flaps

This floor could be lowered (e.g. dropping `for:` to 30s) to detect faster, but that would trade detection speed for **alert noise** — a single benign restart would page an operator. The 2-minute suppression is a conscious signal-vs-noise decision, not an oversight.

5. Root Cause Analysis (5 Whys)

- **Why did a replica set fail to run?** → The container exited immediately on start with a non-zero status.
- **Why did it exit immediately?** → Its startup command was overridden with a malformed sequence (`echo → exit 1`).
- **Why did that cause a crash loop?** → Kubernetes restarts failed containers with exponential backoff; a command that always fails produces an unbounded `CrashLoopBackOff`.
- **Why did the bad command reach the running Deployment?** → It was applied directly via `kubectl patch` (simulating a bad config change / corrupt image entrypoint / bad env in a real deploy).
- **Why can a single bad config take down a whole replica set despite HA?** → Because all replicas are identical — HA protects against *pod/node* failure, **not** against a *bad configuration* that every replica shares. ← **Root cause: configuration errors are not caught before rollout, and are inherently non-redundant (they affect all replicas equally).**

6. What Went Well

- **HA held.** The rolling update strategy meant Kubernetes did not tear down the healthy pods until the new ones were Ready — which never happened — so users saw no downtime. This validated the 2-replica design.
- **Detection worked.** Three complementary alerts fired, correctly describing the failure from different angles (restart rate, scrape failure, not-ready).
- **Clean, fast recovery.** Removing the override triggered an automatic rolling update back to a healthy state in ~11 seconds.

7. What Could Be Improved

- **No pre-rollout config validation.** A bad command reached the cluster with no gate. CI could validate/smoke-test the image before deploy.
- **Detection floor of ~2.5 min** is acceptable but could be reduced for crash loops specifically with a dedicated faster rule if the noise tradeoff is accepted.
- **The for: 2m + rate([5m]) combination**, while correct, means very fast crash loops still take minutes to alert — a `kube_pod_container_status_restarts_total` absolute-threshold rule could complement it.

8. Corrective Actions

#	Action	Owner	Priority	Target Date
1	Document crash-loop response in the runbook (identify, inspect logs, rollback)	Vishnu K	High	2026-08-04
2	Add a CI smoke test that runs the built image and hits <code>/health</code> before deploy	Vishnu K	Medium	Backlog
3	Add a fast complementary alert on absolute restart count (>3 in 5m)	Vishnu K	Low	Backlog
4	Adopt <code>helm --atomic</code> on deploy so failed rollouts auto-rollback	Vishnu K	Medium	Backlog

9. Incidental Finding (unrelated to this incident)

During log review, Loki surfaced a recurring line: `FATAL: database "notes" does not exist`.

- **This is NOT caused by the crash-loop simulation.** It is pre-existing log noise.
- **Root cause identified:** the Postgres readiness/liveness probe runs `pg_isready -U notes` without specifying a database. `pg_isready` then defaults to probing a database named after the user (`notes`), which does not exist (the actual database is `notesdb`). `pg_isready` still returns "accepting connections", so the probe passes — the FATAL line is **harmless probe noise**, not a functional failure.
- **Fix (low priority):** change the probe to `pg_isready -U notes -d notesdb` to eliminate the misleading log line.

- **Why it matters:** flagged rather than ignored — investigating anomalies even when they aren't the current incident is core to good operations.

9. Evidence

Injection & CrashLoopBackOff

```
vishn@VISHNU MINGW64 ~/Downloads
$ ssh -i "sre-test-key.pem" ubuntu@ec2-13-126-63-217.ap-south-1.compute.amazonaws.com
```

```
=====
FAILURE SIMULATION: Pod Crash Loop
Start time: 2026-08-02 22:10:10 UTC
=====
```

[STEP 1] Current pod status (healthy baseline):

NAME	READY	STATUS	RESTARTS	AGE
notes-api-545877b456-7965p	1/1	Running	0	3h21m
notes-api-545877b456-hqjzh	1/1	Running	0	3h22m

[STEP 2] Injecting broken startup command (corrupt config)...

deployment.apps/notes-api patched

[STEP 3] Watching pods enter CrashLoopBackOff (Ctrl+C to stop watching)...

Detection: check Prometheus Alerts for 'NotesApiPodCrashLooping'

Expected alert fire: ~2-3 minutes after crash loop begins

NAME	READY	STATUS	RESTARTS	AGE
notes-api-545877b456-7965p	1/1	Running	0	3h21m
notes-api-545877b456-hqjzh	1/1	Running	0	3h22m
notes-api-756bb4dc5d-cskd9	0/1	Pending	0	0s
notes-api-756bb4dc5d-cskd9	0/1	Pending	0	0s
notes-api-756bb4dc5d-cskd9	0/1	ContainerCreating	0	0s
notes-api-756bb4dc5d-cskd9	0/1	ContainerCreating	0	0s
notes-api-756bb4dc5d-cskd9	0/1	Running	0	1s
notes-api-756bb4dc5d-cskd9	0/1	Error	0	2s
notes-api-756bb4dc5d-cskd9	0/1	Running	1 (1s ago)	2s
notes-api-756bb4dc5d-cskd9	0/1	Error	1 (2s ago)	3s
notes-api-756bb4dc5d-cskd9	0/1	CrashLoopBackOff	1 (2s ago)	4s
notes-api-756bb4dc5d-cskd9	0/1	Running	2 (12s ago)	14s
notes-api-756bb4dc5d-cskd9	0/1	Error	2 (13s ago)	15s
notes-api-756bb4dc5d-cskd9	0/1	CrashLoopBackOff	2 (2s ago)	16s
notes-api-756bb4dc5d-cskd9	0/1	Running	3 (24s ago)	38s
notes-api-756bb4dc5d-cskd9	0/1	Error	3 (25s ago)	39s
notes-api-756bb4dc5d-cskd9	0/1	CrashLoopBackOff	3 (2s ago)	40s

```
ubuntu@ip-172-31-33-133:~/SRE-Project1$ sudo KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get pods -l app=notes-api
```

NAME	READY	STATUS	RESTARTS	AGE
notes-api-545877b456-7965p	1/1	Running	0	3h6m
notes-api-545877b456-hqjzh	1/1	Running	0	3h6m
notes-api-756bb4dc5d-mlf6z	0/1	CrashLoopBackOff	4 (22s ago)	115s

```
ubuntu@ip-172-31-33-133:~/SRE-Project1$
```

Alert firing

NotesApiPodCrashLooping

rate(kube_pod_container_status_restarts_total{namespace="default",pod=~"notes-api.*"}[5m]) > 0

for: 2m

severity: "critical"

description

Pod [{{ \$labels.pod }}] has restarted repeatedly in the last 5 minutes.

summary

Notes API pod is crash-looping

Alert labels

alertname="NotesApiPodCrashLooping"	app="notes-api"	container="notes-api"	endpoint="http"	instance="10.42.0.53:8080"	job="kubernetes-metrics"	namespace="default"	pod="notes-api-75bb4dc5d-c4d9"	service="monitoring-kube-state-metrics"	severity="critical"
State: FIRING Active Since: 4m 3.732s Value: 0.011439999999999999									

NotesApiHighCPU

NotesApiPodNotReady

NotesApiDown

up{job="notes-api"} == 0

for: 1m

severity: "critical"

description

Prometheus cannot scrape Notes API instance [{{ \$labels.instance }}].

summary

Notes API target is down

Alert labels

alertname="NotesApiDown"	app="notes-api"	container="notes-api"	endpoint="http"	instance="10.42.0.76:8080"	job="notes-api"	namespace="default"	pod="notes-api-75bb4dc5d-c4d9"	service="notes-api"	severity="critical"
State: FIRING Active Since: 4m 36.139s Value: 0									

Logs (Loki)

Logs

1000 lines shown - 26.49% (15min 54sec) of 1h Total bytes processed: 361 kB Common labels: namespace=default node_name=ip-172-31-35-133

Search fields by name

Show log level

Fields

app

container

filename

job

namespace

node_name

pod

stream

2026-08-03 03:28:51.646 10.42.0.1 - - [02/Aug/2026:21:58:51 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:48.671 10.42.0.1 - - [02/Aug/2026:21:58:48 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:47.560 10.42.0.1 - - [02/Aug/2026:21:58:47 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:47.071 2026-08-02 21:58:47.071 UTC [28951] FATAL: database "notes" does not exist

2026-08-02 21:58:46.802 2026-08-02 21:58:46.802 UTC [28944] FATAL: database "notes" does not exist

2026-08-03 03:28:46.744 10.42.0.1 - - [02/Aug/2026:21:58:46 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:46.646 10.42.0.1 - - [02/Aug/2026:21:58:46 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:45.204 10.42.0.55 - - [02/Aug/2026:21:58:45 +0000] "GET /metrics HTTP/1.1" 200 147815 "-" "Prometheus/3.13.2"

2026-08-03 03:28:44.880 10.42.0.55 - - [02/Aug/2026:21:58:44 +0000] "GET /metrics HTTP/1.1" 200 14177 "-" "Prometheus/3.13.2"

2026-08-03 03:28:43.670 10.42.0.1 - - [02/Aug/2026:21:58:43 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-02 21:58:42.070 2026-08-02 21:58:42.070 UTC [28935] FATAL: database "notes" does not exist

2026-08-03 03:28:41.646 10.42.0.1 - - [02/Aug/2026:21:58:41 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:38.671 10.42.0.1 - - [02/Aug/2026:21:58:38 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:37.559 10.42.0.1 - - [02/Aug/2026:21:58:37 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-02 21:58:37.069 2026-08-02 21:58:37.069 UTC [28927] FATAL: database "notes" does not exist

2026-08-02 21:58:36.801 2026-08-02 21:58:36.801 UTC [28920] FATAL: database "notes" does not exist

2026-08-03 03:28:36.744 10.42.0.1 - - [02/Aug/2026:21:58:36 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:36.646 10.42.0.1 - - [02/Aug/2026:21:58:36 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:36.513 FATAL: corrupt startup configuration

2026-08-03 03:28:33.670 10.42.0.1 - - [02/Aug/2026:21:58:33 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-02 21:58:32.071 2026-08-02 21:58:32.071 UTC [28913] FATAL: database "notes" does not exist

2026-08-03 03:28:31.646 10.42.0.1 - - [02/Aug/2026:21:58:31 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:30.187 10.42.0.55 - - [02/Aug/2026:21:58:30 +0000] "GET /metrics HTTP/1.1" 200 10834 "-" "Prometheus/3.13.2"

2026-08-03 03:28:29.899 10.42.0.55 - - [02/Aug/2026:21:58:29 +0000] "GET /metrics HTTP/1.1" 200 140662 "-" "Prometheus/3.13.2"

2026-08-03 03:28:28.670 10.42.0.1 - - [02/Aug/2026:21:58:28 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:27.560 10.42.0.1 - - [02/Aug/2026:21:58:27 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-02 21:58:27.070 2026-08-02 21:58:27.070 UTC [28904] FATAL: database "notes" does not exist

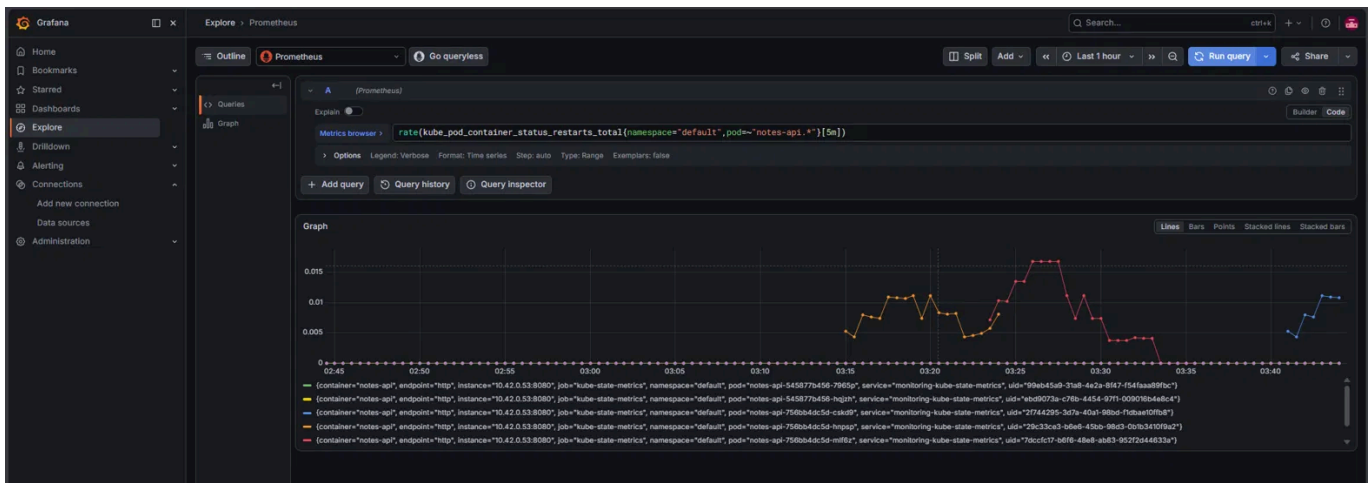
2026-08-02 21:58:26.801 2026-08-02 21:58:26.801 UTC [28897] FATAL: database "notes" does not exist

2026-08-03 03:28:26.744 10.42.0.1 - - [02/Aug/2026:21:58:26 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

2026-08-03 03:28:26.646 10.42.0.1 - - [02/Aug/2026:21:58:26 +0000] "GET /health HTTP/1.1" 200 87 "-" "kube-probe/1.36"

Metric impact

5 / 6



Recovery

```
^Cubuntu@ip-172-31-33-133:~/SRE-Project1/tests/simulate-pod-crashloop.sh --recover
echo "RECOVERY: $(date -u '+%Y-%m-%d %H:%M:%S UTC')"
```

```
[RECOVERY] Removing the broken command override...
deployment.apps/notes-api patched
[RECOVERY] Waiting for rollout to stabilize...
deployment "notes-api" successfully rolled out
[RECOVERY] Done. Pods should be healthy again.
RECOVERY: 2026-08-02 22:16:19 UTC
ubuntu@ip-172-31-33-133:~/SRE-Project1$
```

11. Reproduction

```
ssh ubuntu@13.126.63.217
cd ~/SRE-Project1
./tests/simulate-pod-crashloop.sh
# wait ~3 min, observe NotesApiPodCrashLooping firing in Prometheus
./tests/simulate-pod-crashloop.sh --recover
```